

Functional Programming

Radu Răzvan Slăvescu

Technical University of Cluj-Napoca
Department of Computer Science

Outline

Functions as data

(.) for function composition

map **and** filter revisited

fold family

takeWhile **and** alike

Remember: functions

Remark: First class citizenship for functions

In FP, **functions** could be regarded even as **data**

1. functions can be arguments or results of functions
2. functions can be elements of lists

First order functions versus higher-order functions

Definition (First order function)

Its arguments and returned result are both **data**

Definition (Higher-order function, aka functional)

Its arguments, returned result, or both, are **functions**

Remarks: advantages of Higher-order functions

1. One can add programming idioms as functions into the language itself hence higher abstraction and reuse
2. Their algebraic properties help in reasoning on programs

Functions as data

Example (A function sent as a parameter)

```
myinser x [] _ = [x]
myinser x (y:ys) orel
  | orel x y = x:y:ys
  | otherwise= y:myinser x ys orel

mysortins [] _ = []
mysortins (x:xs) orel =
  myinser x (mysortins xs orel) orel

f a b = a<b
```

Functions as data

Example (A function sent as a parameter)

```
*Main> mysortins [3,1,2,1,4] f  
[1,1,2,3,4]
```

```
*Main> mysortins [3,1,2,1,4] (<)  
[1,1,2,3,4]
```

```
*Main> mysortins [3,1,2,1,4] (\x y -> x>=y)  
[4,3,2,1,1]
```

Remark

Here, `orel` is a function which compares two entities; it is passed to `inser` and tells in which order the list will be sorted

Functions as data

Example (A list of functions)

```
processWage :: [t -> t] -> t -> t
processWage [] w = w
processWage lop w = processWage (tail lop)
                      (head lop w)
```

Extract an element from a list of functions and use it:

```
*Main> processWage [\ w -> w - 25.00,
                  \ w -> 0.65 * w,
                  \ w -> 0.90 * w] 1025.00
585.0
```

When are two functions equal?

Function equality: when are two functions equal?

(i) same expressions v. (ii) same results for identical arguments

Definition (Extensional equality of functions)

f and g are extensionally equal ($f = g$) iff $\forall x : f\ x = g\ x$

Example (Functions as data)

```
*Main> i1 x = x+x -- same as i1 = \x -> x+x
*Main> i2 x = 2*x -- extensionally equal to i1
*Main> i3 = i1 -- "copy" body of i1
*Main> i1 4
8
*Main> i2 4
8
```


Functions as data

Example (Functions as data)

```
*Main> i3 4
```

```
8
```

```
*Main> i1 x = x+x+1
```

```
*Main> i1 4
```

```
9
```

```
*Main> i3 4
```

```
8
```

The functional composition

Definition (Function composition: $(f \cdot g) x = f (g x)$)

```
(.) :: (b -> c) -> (a -> b) -> a -> c  
f . g = \ x -> f (g x)
```

Example (Functional composition)

```
Main> map (not . odd) [1..5]  
[False,True,False,True,False]
```

```
odd      :: Integral a => a -> Bool  
not      :: Bool -> Bool  
not . odd :: Integral a => a -> Bool  
(.)      :: (b -> c) -> (a -> b) -> a -> c
```

The functional composition

Example (Functional composition)

```
odd      :: Integral a => a -> Bool
not      :: Bool -> Bool
not . odd :: Integral a => a -> Bool
(.) :: (b -> c) -> (a -> b) -> a -> c
```

Remark

Functions `odd`, `not` and `not . odd` are first order. On the other hand, `(.)` is a higher-order function (or operator). It builds a new, nameless function from its operands `not` and `odd`, which is then sent as an argument to `map`.

The functional composition

Example (Function composition exercise)

Guess (1) Type; (2) example call (3) reasonable name for:

```
((length . ) . filter)
```

Function application

Definition (Function application operator)

```
infixr 0 $  
($) :: (a -> b) -> a -> b  
f $ x = f x
```

Example

```
sum (map (^2) (filter (>0) [1,2,-2,3]))
```

same as with this more readable one:

```
sum $ map (^2) $ filter (>0) [1,2,-2,3]
```

Remember: The higher-order function `map`

Definition (Map)

It applies a given function on every element of a list.

Example (Haskell `map`)

```
Prelude> map (\ x -> x*x) [1,2,3]  
[1,4,9]
```

Example (Defining `map` using list comprehensions)

```
mapGen :: (a->b) -> [a] -> [b]  
mapGen f xs = [ f x | x <- xs]
```

Remember: The higher-order function `filter`

Definition (Filter)

It selects the elements of a list satisfying a given predicate.

Example (Haskell `filter`)

```
Main> filter odd [1..5]
[1,3,5]
```

Example (Defining `filter` using list comprehensions)

```
filterGen :: (a -> Bool) -> [a] -> [a]
filterGen p xs = [ x | x <- xs, p x]
```

The higher-order functions `map` and `filter`

Remember

- ▶ both `map` and `filter` iterate structures
- ▶ they separate structure traversal from data item processing

Remember: The higher-order function `foldr`

Remember `foldr`

Many functions are defined using the following pattern:

$$\begin{aligned}f [] &= v \\f (x : xs) &= x \oplus f xs\end{aligned}$$

Function `foldr :: (a -> b -> b) -> b -> [a] -> b` would incorporate this pattern, with the $\oplus$ operation and the `v` value as parameters. Operator $\oplus$ should be right associative.

Remember: some higher-order functions on lists

Example (Haskell `foldr`)

```
Prelude> foldr (*) 1 [1,2,3]  
6
```

Example (New definitions for `sum` and `and`)

```
mynewsum = foldr (+) 0
```

```
mynewand = foldr (&&) True
```

Understanding `foldr`

`foldr`

One can regard `foldr` as simultaneously replacing `(:)` in a list by $\oplus$ and `[]` by v . The $\oplus$ operator should be **right associative**.

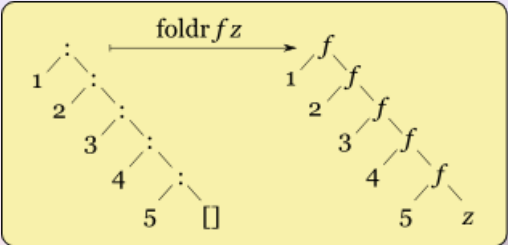
Example

```
mynewsum [1,2,3] =  
foldr (+) 0 [1,2,3] =  
foldr (+) 0 (1:(2:(3:[]))) =  
                (1+(2+(3+0))) =  
6
```

Example (But what if we do this?)

```
*Main> foldr (-) 0 [1,2,3]  
2
```

Understanding `foldr` (Image source: wiki.haskell.org)



Replace (:) by $\mathbb{f}$ and [] by $\mathbb{v}$

Some examples on lists

Example (Reverse)

```
myrev [] = []
```

```
myrev (x:xs) = myrev xs ++ [x]
```

```
myrev [1,2,3] =
```

```
myrev (1:(2:(3:[]))) =
```

```
(([] ++ [3]) ++ [2]) ++ [1] =
```

```
[3,2,1]
```

```
mynewrev = foldr (\ x xs -> xs ++ [x]) []
```

Some more examples on lists

Example (Member)

```
mynewmember e xs = foldr (||) False  
                  (map ((==) e) xs)
```

Remark

Here, we `map` the equality-to-`e`-test on list `xs` and get a list of `True` and `False` values with the same length as `xs`; folding `(||)` on it tells us whether there is at least one `True` inside.

Example (Append)

```
mynewappend xs = foldr (:) xs
```

Example

```
mynewflatten = foldr (++) []
```

All and any

Example (All: every element satisfies a predicate)

```
mynewall p xs = foldr (&&) True (map p xs)
```

```
Main> mynewall odd [1,3]
```

```
True
```

```
Main> mynewall odd [1,2,3]
```

```
False
```

Example (Any: at least one element satisfies a predicate)

```
mynewany p xs = foldr (||) False (map p xs)
```

The higher-order function *foldl*

`foldl`

Many functions are defined using the following pattern:

$$\begin{aligned} f\ v\ [] &= v \\ f\ v\ (x : xs) &= f\ (v \oplus x)\ xs \end{aligned}$$

In Haskell, this pattern is incorporated by the function

```
foldl :: (a -> b -> a) -> a -> [b] -> a
```

It has $\oplus$ operation and the v value as first two parameters.

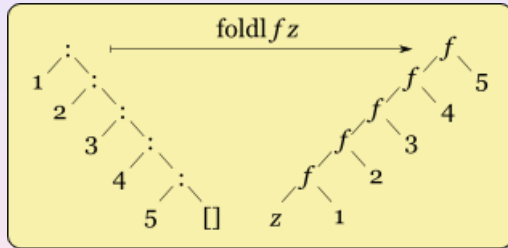
$\oplus$ should be **left associative**.

Understanding `foldl`

Haskell `foldl`

$$\begin{aligned} \text{foldl } (\oplus) \ v \ [x_0, x_1, \dots, x_n] &= \\ (((v \oplus x_0) \oplus x_1) \oplus \dots) \oplus x_n &= \\ (((v \oplus x_0) \oplus x_1) \oplus \dots) \oplus x_n &= \\ \dots = \\ (((v \oplus x_0) \oplus x_1) \oplus \dots) \oplus x_n \end{aligned}$$

Understanding `foldl` (Image source: wiki.haskell.org)



Reflect tree
Reverse list
Replace `(:)` and `[]`

foldr v. foldl

Example

```
Prelude> foldr (++) "r"  
["o", "t", "t", "e"]  
"otter"
```



Example

```
Prelude> foldl (++) "r"  
["o", "t", "t", "e"]  
"rotte"
```



foldr v. foldl

Remember

Haskell operator $(^)$ is right associative: $4^3^2 = 4^3^2 = 4^3^2$
(i.e., $4^{3^2} = 4^{(3^2)} = 262144$, as opposite to $(4^3)^2 = 4096$)

Example (Your turn)

```
Prelude> foldr (^) 1 [2,2,2]
```

```
16
```

```
Prelude> foldl (^) 1 [2,2,2]
```

```
1
```

```
Prelude> foldl (^) 2 [2,2,2]
```

```
256
```

```
Prelude> foldr (^) 2 [2,2,2]
```

```
65536
```

foldr v. foldl

Remark

1. `foldl` is related to tail recursion
2. sometimes, `foldr` can give the answer, but `foldl` cannot

Example

```
*Main> foldr (&&) True [ False, False ..]  
False  
*Main> foldl (&&) True [ False, False ..]  
^CInterrupted.
```

Understanding foldl

Elm foldl

$$\begin{aligned} \text{foldl } (\oplus) \ v \ [x_0, x_1, \dots, x_n] &= \\ x_n \oplus (\dots (x_1 \oplus (x_0 \oplus v)) \dots) &= \\ x_n \oplus (\dots (x_1 \oplus (x_0 \oplus v)) \dots) &= \\ \dots &= \\ x_n \oplus (\dots (x_1 \oplus (x_0 \oplus v)) \dots) \end{aligned}$$

Understanding foldl

Example

```
> import List as L
L.foldl (+) 0 [1,2,3] =
L.foldl (+) 0 (1:(2:(3:[]))) =
    3+(2+(1+0)) =
6
```

Example (But what if we do this?)

```
*Main> L.foldl (-) 0 [1,2,3]
2
```

Understanding `foldl`

Haskell-style v. Elm-style `foldl`

Elm preserves the **signature** of `foldr`:

```
> List.foldr
<function> : (a -> b -> b) -> b -> List a -> b
> List.foldl
<function> : (a -> b -> b) -> b -> List a -> b
```

Haskell **swaps** the parameters of the operation.

```
Prelude> :t foldr
foldr :: (a -> b -> b) -> b -> [a] -> b
Prelude> :t foldl
foldl :: (b -> a -> b) -> b -> [a] -> b
```


Your turn

Example (Warming up)

```
ra = foldl (\ xs x -> x:xs) []
```

```
Main> ra [1,2,3]
```

```
[3,2,1]
```

Your turn

Example (Yet another one)

```
f1210 = foldl (\ y x -> x+2*y) 0
```

```
Main> f1210 [1,0,1,1]
```

```
11
```

Example (And the last one)

```
f :: [Integer] -> Integer
```

```
f l = foldr (\ x y -> x+2*y) 0 (reverse l)
```

```
Main> f [1,0,1,1]
```

```
11
```

foldl'

Remark

A strict version of `foldl`.

Example

```
Prelude> import Data.List
Prelude Data.List> foldl' (+) 0 [1..100000000]
500000000500000000
(2.67 secs, 8,800,076,312 bytes)
Prelude Data.List> foldr (+) 0 [1..100000000]
*** Exception: stack overflow
```

Remark

Usually, you'll have to choose between `foldr` and `foldl'`.

foldl1

Definition

```
foldl1 :: (a -> a -> a) -> [a] -> a  
foldl1 f (x:xs) = foldl f x xs
```

Remark

Similar to `foldl`, but the argument cannot be an empty list.

Example (Lowest value in a list: Haskell `minimum`)

```
minimum :: Ord a => [a] -> a  
minimum = foldl1 min
```

`min :: Ord a => a -> a -> a` returns the lowest of its two arguments

foldr1

Definition

```
foldr1 :: (a -> a -> a) -> [a] -> a
foldr1 _ [x] = x
foldr1 f (x:xs) = f x (foldr1 f xs)
```

Remark

Similar to `foldr`, but the argument cannot be an empty list.

Example (Haskell `unwords`)

```
*Main> foldr1 (\s ss -> s ++ " " ++ ss)
           ["no", "trailing", "space"]
"no trailing space"
```

scanl

Definition

```
myscanl :: (b -> a -> b) -> b -> [a] -> [b]
myscanl f v []      = [v]
myscanl f v (x:xs) = v : myscanl f (f v x) xs
```

Remark

Similar to `foldl`, but returns the list of partial values from left.

Example

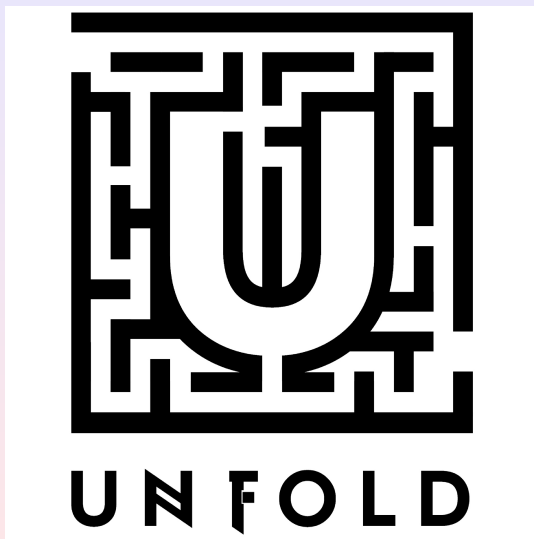
```
Main> scanl (+) 0 [10, 12, 15]
[0,10,22,37]
```

The power of fold

Example

```
reversef xs = foldl (\ xs x -> x:xs) [] xs
lengthf xs = foldl (\ n _ -> n+1) 0 xs
mapf f xs = foldr (\ y ys -> (f y):ys) [] xs
filterf p xs =
    foldr (\ y ys -> if (p y) then y:ys else ys) []
    xs
```

Some more higher-order functions



Some more higher-order functions

unfold with arguments p , h , t , x

```
unfold p h t x
| p x      = []
| otherwise = (h x) : unfold p h t (t x)
```

If $p\ x$ is true, then return an empty list (end of story)
else put $h\ x$ as the first element of the result and
call $t\ x$ to generate the rest of the result,
which will be processed in a similar manner

Example (unfold)

```
*Main> unfold (==0) ('mod' 2) ('div' 2) 11

[1,1,0,1]
```

Some more higher-order functions

unfold with arguments p , h , t , x

```
unfold p h t x
| p x          = []
| otherwise = (h x) : unfold p h t (t x)
```

Example (Your turn)

```
unfold (null) ((2*) . head) (tail) [1,2,3]
```

Some more higher-order functions

takeWhile

Select elements of a list as long as they satisfy a predicate.

```
mytakeWhile :: (a -> Bool) -> [a] -> [a]
```

```
mytakeWhile _ [] = []
```

```
mytakeWhile p (x:xs)
```

```
  | p x = x:mytakeWhile p xs
```

```
  | otherwise = []
```

Example (takeWhile)

```
*Main> mytakeWhile (odd) [1,3,5,6,7,8]  
[1,3,5]
```

Some more higher-order functions

dropWhile

Remove elements of a list as long as they satisfy a predicate.

Example (dropWhile)

```
*Main> mydropWhile (odd) [1,3,5,6,7,8]  
[6,7,8]
```

Some more higher-order functions

span

Split a list at the first element **not** satisfying a predicate.

```
myspan p xs = (takeWhile p xs, dropWhile p xs)
```

Example (span)

```
*Main> myspan (odd) [1,3,5,6,7,8]  
([1,3,5],[6,7,8])
```

span v. splitAt

mysplitAt

Split a list at a given index

```
mysplitAt n xs = (take n xs, drop n xs)
```

Example (mysplitAt)

```
*Main> mysplitAt 2 [1,3,5,6,7,8]  
([1,3],[5,6,7,8])
```

Using higher-order functions

Example (Your turn)

```
i xs ys = filter (combC elem ys) xs where  
  combC f x y = f y x
```

Example (Your turn)

```
l xs = foldr (+) 0 (map (combK 1) xs) where  
  combK k _ = k
```

Example (Your turn)

```
ll = (foldr (+) 0) . (map (combK 1)) where  
  combK k _ = k
```

Using higher-order functions

Example (Your turn)

```
l1l :: [Int] -> Int  
l1l = foldr (\_ n -> 1+n) 0
```

Example (Your turn)

```
m f = foldr ( (:) . f ) []
```


Using higher-order functions

Example (Your turn)

```
li = "Now I need a drink, alcoholic sugarfree of  
course, after the heavy lectures involving  
magical functions, and pi, and formulas, when  
people go really mad but thriving "
```

```
p = map length . hew [] . filter (/='(',')') where  
  hew a "" = reverse a  
  hew a cs = let  
                (f,s) = span (/=' ') cs  
              in  
                hew (f:a) (tail s)
```

Summary

- ▶ in FP, data and functions can be regarded in the same way
- ▶ `map` and `filter` iterate structures and separate structure traversal from element processing
- ▶ `fold` family members combine elements of a data structure, according to a given operator, into one value